

라즈베리파이로 YOLO 사용해보기



Contents

1 YOLO?

1.1 You Only Look Once

2 YOLO

2.1 실습 환경 확인

2.2 패키지 설치

2.3 라이브러리 불러오기

2.4 카메라 사용 방식

2.5 카메라 한 장 테스트

2.6 YOLO11n 모델 불러오기

2.7 카메라 사진 한 장에 YOLO11 적용하기

2.8 탐지 결과 읽기

2.9 짧은 실시간 탐지 실행하기

2.10 입력 크기로 속도 조절하기

2.11 탐지 결과 이미지 저장하기

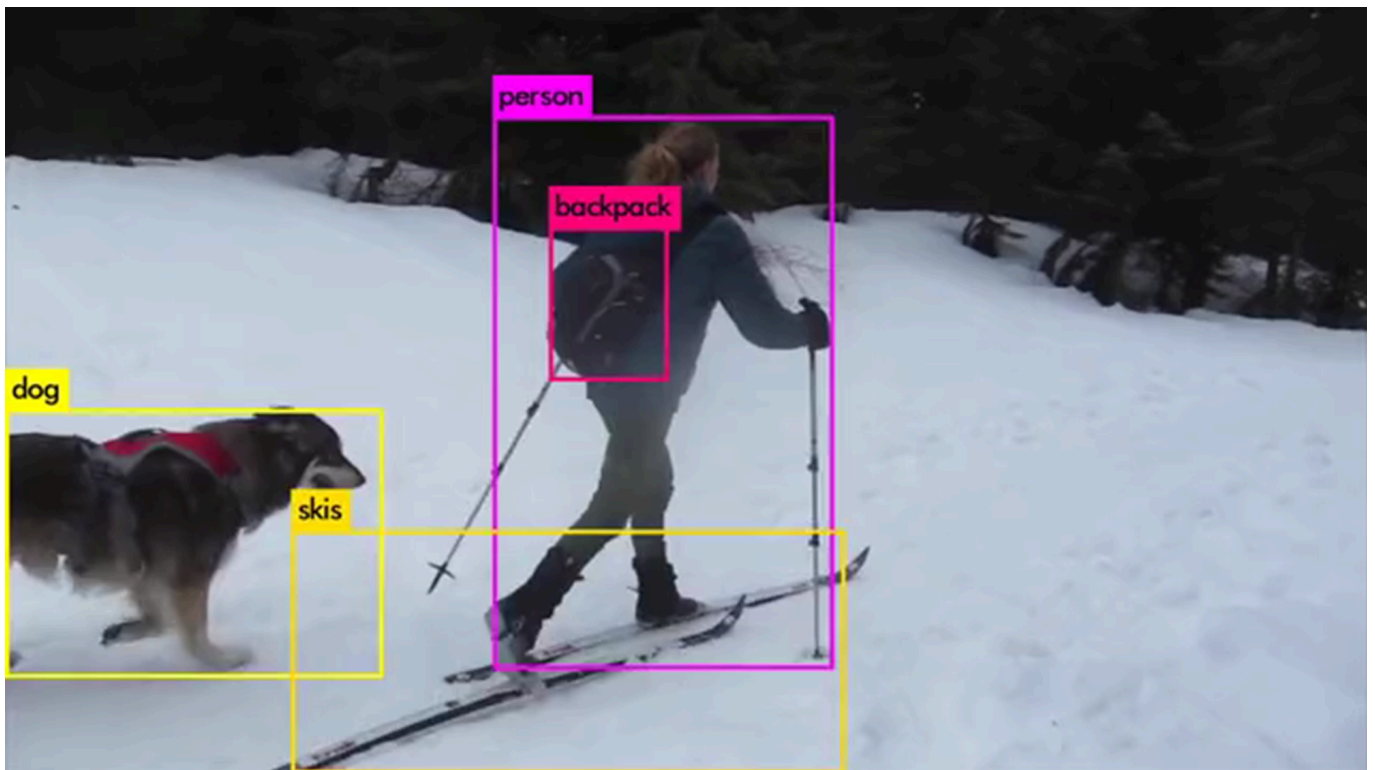
라즈베리파이로 YOLO 사용해보기

Exported from Confluence · <https://pinkwink.atlassian.net>

YOLO?

You Only Look Once (<https://docs.ultralytics.com/ko>)

이미지 내의 여러 객체를 한 번의 추론으로 식별하고 위치를 파악하는 알고리즘



YOLO

실습 환경 확인

먼저 현재 Python 버전과 실행 환경을 확인한다.

```
[1]: import sys, platform, os

print("Python:", sys.version)
print("Platform:", platform.platform())
print("Working directory:", os.getcwd())

Python: 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
Platform: Linux-6.8.0-1040-raspi-aarch64-with-glibc2.39
Working directory: /home/pinky/my_petbot
```

패키지 설치

YOLO11 실행에는 `ultralytics`, 이미지 표시에는 `pillow`, 영상 배열 처리를 위해 `opencv-python` 을 사용

```
[3]: # 필요한 패키지 설치
# 이미 설치되어 있다면 이 셀은 건너뛰어도 됩니다.
!python3 -m pip install -U pip
!python3 -m pip install ultralytics pillow opencv-python

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in /home/pinky/.local/lib/python3.12/site-packages (25.3)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
----- 1.8/1.8 MB 4.6 MB/s 0:00:00 eta 0:00:01
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.3
    Uninstalling pip-25.3:
      Successfully uninstalled pip-25.3
  WARNING: The scripts pip, pip3 and pip3.12 are installed in '/home/pinky/.local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed pip-26.1.1
Defaulting to user installation because normal site-packages is not writeable
Collecting ultralytics
  Downloading ultralytics-8.4.50-py3-none-any.whl.metadata (39 kB)
Requirement already satisfied: pillow in /usr/lib/python3/dist-packages (10.2.0)
Requirement already satisfied: opencv-python in /home/pinky/.local/lib/python3.12/site-packages (4.10.0.84)
Requirement already satisfied: numpy>=1.23.0 in /home/pinky/.local/lib/python3.12/site-packages (from ultralytics) (1.26.4)
Requirement already satisfied: matplotlib>=3.3.0 in /home/pinky/.local/lib/python3.12/site-packages (from ultralytics) (3.9.2)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/lib/python3/dist-packages (from ultralytics) (6.0.1)
Requirement already satisfied: requests>=2.23.0 in /usr/lib/python3/dist-packages (from ultralytics) (2.31.0)
Requirement already satisfied: scipy>=1.4.1 in /usr/lib/python3/dist-packages (from ultralytics) (1.11.4)
Collecting torch>=1.8.0 (from ultralytics)
  Downloading torch-2.12.0-cp312-cp312-manylinux_2_28_aarch64.whl.metadata (31 kB)
Collecting torchvision>=0.9.0 (from ultralytics)
  Downloading torchvision-0.27.0-cp312-cp312-manylinux_2_28_aarch64.whl.metadata (5.5 kB)
Requirement already satisfied: psutil>=5.8.0 in /usr/lib/python3/dist-packages (from ultralytics) (5.9.8)
Collecting polars>=0.20.0 (from ultralytics)
  Downloading polars-1.40.1-py3-none-any.whl.metadata (10 kB)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.19-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in /home/pinky/.local/lib/python3.12/site-packages (from matplotlib>=3.3.0->ultralytics) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /home/pinky/.local/lib/python3.12/site-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/pinky/.local/lib/python3.12/site-packages (from matplotlib>=3.3.0->ultralytics) (4.55.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/pinky/.local/lib/python3.12/site-packages (from matplotlib>=3.3.0->ultralytics) (1.4.7)
```

라이브러리 불러오기

```
In [ ]: 1 from ultralytics import YOLO
2 from PIL import Image
3 from IPython.display import display, clear_output
4 import time
5 import io
6 import os
7
8 # 업로드한 my_petbot.ipynb에서 사용한 카메라 방식
9 from pinkylib import Camera
```

카메라 사용 방식

한 장의 프레임을 가져온다.

```
In [ ]: 1 def capture_one_frame(cam):
2     """pinkylib.Camera에서 프레임 한 장을 가져옵니다.
3
4     업로드한 노트북 기준:
5     - cam.get_frame()으로 프레임 획득
6     - 프레임은 BGR 순서
7     - 화면 표시용으로는 RGB로 변환
8     """
9     frame_bgr = cam.get_frame()
10    image_rgb = Image.fromarray(frame_bgr[:, :, ::-1])
11    return frame_bgr, image_rgb
12
13 14 def capture_one_frame_bytes(cam):
15     """카메라 프레임을 JPEG bytes로 변환합니다.
16
17     OpenAI Vision API나 파일 저장 등에 사용할 수 있는 형태입니다.
18     """
19     frame_bgr, image_rgb = capture_one_frame(cam)
20     buf = io.BytesIO()
21     image_rgb.save(buf, format="JPEG")
22     return buf.getvalue()
```

카메라 한 장 테스트

```
In [ ]: 1 def test_camera_once():
2     cam = Camera()
3     cam.start()
4     try:
5         frame_bgr, image_rgb = capture_one_frame(cam)
6         display(image_rgb)
7         return frame_bgr
8     finally:
9         cam.close()
10
11 last_frame = test_camera_once()
```

YOLO11n 모델 불러오기

처음 실행할 때는 모델 파일이 자동으로 다운로드

```
In [ ]: 1 model = YOLO("yolo11n.pt")
        2 print("Model loaded:", model.model_name if hasattr(model, "model_name") else "yo
```

카메라 사진 한 장에 YOLO11 적용하기

```
In [ ]: 1 def detect_once_from_camera(imgsz=640, conf=0.25):
        2     cam = Camera()
        3     cam.start()
        4     try:
        5         frame_bgr, image_rgb = capture_one_frame(cam)
        6         display(image_rgb)
        7
        8         results = model.predict(
        9             source=frame_bgr,
        10             imgsz=imgsz,
        11             conf=conf,
        12             verbose=False
        13         )
        14
        15         annotated_bgr = results[0].plot()
        16         annotated_rgb = Image.fromarray(annotated_bgr[:, :, ::-1])
        17         display(annotated_rgb)
        18
        19         return results[0], annotated_bgr
        20     finally:
        21         cam.close()
        22
        23 last_result, last_annotated_bgr = detect_once_from_camera(imgsz=640, conf=0.25)
```

탐지 결과 읽기

YOLO 결과에는 물체 이름, 신뢰도, 박스 좌표가 들어 있다.

```
[10]: def print_detection_result(result):
    if result.bboxes is None or len(result.bboxes) == 0:
        print("탐지된 물체가 없습니다.")
        return

    for i, box in enumerate(result.bboxes, start=1):
        cls_id = int(box.cls[0])
        name = model.names[cls_id]
        conf = float(box.conf[0])
        x1, y1, x2, y2 = [float(v) for v in box.xyxy[0]]
        print(f"{i}. {name} / confidence={conf:.2f} / box=({x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f})")

print_detection_result(last_result)
```

탐지된 물체가 없습니다.

짧은 실시간 탐지 실행하기

카메라 프레임을 여러 번 가져와 YOLO11을 반복 실행하면 짧은 실시간 객체 탐지처럼 사용할 수 있다.

```
In [ ]: 1 def run_short_realtime_detection(num_frames=30, imgsz=320, conf=0.25, delay=0.03)
2         cam = Camera()
3         cam.start()
4         try:
5             for i in range(num_frames):
6                 frame_bgr, _ = capture_one_frame(cam)
7                 results = model.predict(
8                     source=frame_bgr,
9                     imgsz=imgsz,
10                    conf=conf,
11                    verbose=False
12                )
13
14                annotated_bgr = results[0].plot()
15                annotated_rgb = Image.fromarray(annotated_bgr[:, :, ::-1])
16
17                clear_output(wait=True)
18                print(f"frame {i + 1}/{num_frames} | imgsz={imgsz} | conf={conf}")
19                display(annotated_rgb)
20
21                time.sleep(delay)
22            finally:
23                cam.close()
24
25        run_short_realtime_detection(num_frames=30, imgsz=320, conf=0.25)
```

입력 크기로 속도 조절하기

입력 크기인 `imgsz` 를 줄이면 작은 물체를 놓칠 가능성이 커지지만, 속도가 빨라질 수 있다

```
In [ ]: 1 def compare_imgsz_once():
2         cam = Camera()
3         cam.start()
4         try:
5             frame_bgr, image_rgb = capture_one_frame(cam)
6             display(image_rgb)
7         finally:
8             cam.close()
9
10        for size in [320, 480, 640]:
11            t0 = time.perf_counter()
12            result = model.predict(source=frame_bgr, imgsz=size, conf=0.25, verbose=
13            elapsed = time.perf_counter() - t0
14            fps = 1 / elapsed if elapsed > 0 else 0
15            print(f"imgsz={size} | time={elapsed:.3f} sec | approx FPS={fps:.2f} | de
16
17        compare_imgsz_once()
```

탐지 결과 이미지 저장하기

저장된 이미지는 `yolo11_result.jpg` 라는 이름으로 현재 폴더에 만들어진다.

```
In [ ]: 1 # 마지막 탐지 결과 이미지 저장
2 # OpenCV 없이도 PIL로 저장할 수 있도록 RGB로 변환해서 저장합니다.
3 if "last_annotated_bgr" in globals():
4     save_img = Image.fromarray(last_annotated_bgr[:, :, ::-1])
5     save_path = "yolo11_result.jpg"
6     save_img.save(save_path)
7     print("Saved:", os.path.abspath(save_path))
8     display(save_img)
9 else:
10    print("먼저 '카메라 사진 한 장에 YOLO11 적용하기' 셀을 실행하세요.")
```




PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio